Project 2: Windows

DUE: Tuesday, October 13th at 11:59pm
This is an individual effort assignment
Extra Credit Available for Early Submissions!

Basic Procedures

You must:

- Pass the provided style checker from P0 on all your Java files.
- Comment your code well IN ADDITION to passing the provided JavaDoc checker from P0 on all your Java files for this project (no need to overdo it, just do it well).
- Have code that compiles with the command: `javac *.java` in your code directory without errors or warnings.
- For methods that come with a big-O requirement (check the provided template Java files for details), make sure your implementation meets the requirement.
- Have code that runs with the command: `java WindowManager`

You may:

- Add additional methods and class/instance variables; however, they **must be private**.

You may NOT:

- Make your program part of a package.
- Add additional public methods or public class/instance variables, remember that local variables are not the same as class/instance variables!
- Use any built in Java Collections Framework classes anywhere in your program (e.g. no ArrayList, LinkedList, HashSet, etc.).
- **Create or use any arrays anywhere in your program. You *may not* call `toArray()` methods to bypass this requirement.**
- Alter any method signatures defined in this document of the template code. Note: "throws" is part of the method signature in Java, don't add/remove these.
- Alter provided classes that are complete (e.g. `WindowManager`).
- Add any additional import statements (or use the "fully qualified name" to get around adding import statements).
- Add any additional libraries/packages which require downloading from the internet.
- Use internet or AI assistance to write your solution (or part of your solution).
- Copy/Paste code from your textbook or lecture notes. See the syllabus for detailed information about Academic Standards.

Setup

- Create a folder for `p2`. This is your "project directory".
- Download the `p2.zip` and unzip it in your project directory. This will create a folder `yourCodeHere`. This is your "code directory".
- Move the style/Javadoc checker from P0 to P2 by taking the following files: `checkstyle.jar`, `cs310code.xml`, and `cs310comments.xml`, and put them in your project directory.

Grading Rubric

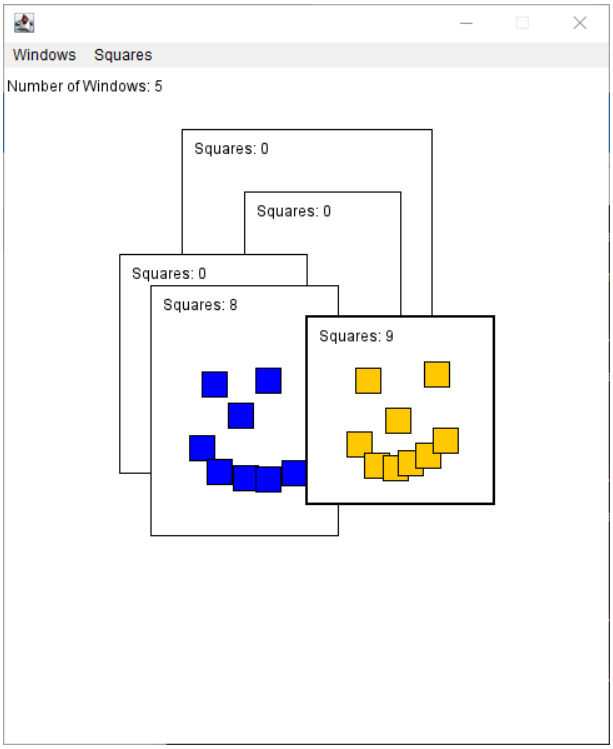
Due to the complexity of this assignment, an accompanying grading rubric pdf has been included with this assignment. Please refer to this document for a complete explanation of the grading.

Submission Instructions

- Go to GradeScope and submit ***just the Java files from your CODE directory*** to the Project 2 submission link.
- You can resubmit as many times as you'd like; only the last submission will be graded.
- ***The GradeScope autograder will give you some immediate feedback*** to ensure you have submitted everything correctly. READ THIS FEEDBACK.

Overview

You're going to build a window manager! This program will work like a traditional window manager, but very limited.



Things your application will be able to do by the end:

- Draw on the window/region with "squares" (see image).
- Remove squares from windows.
- Move windows into the foreground.
- Remove windows.
- Add new windows.

There are five major components to this project:

1. Implementing squares.
2. Implementing collections of squares.
3. Implementing windows.
4. Implementing stacks of windows.
5. Implementing the menus (sorting windows/squares).

It is recommended that you do the project in the order listed above, since each part depends on previous work.

Testing

Before you start reading the project further, and especially before writing any code, you need to be aware that there are no "yays" tester for this project. Part of this project is about you learning to test your own work and verify it is correct. So how should you test?

1. You can always add main methods to classes that don't already have them, and you can write your own "yays". Remember to either add JavaDocs to these or remove them before submission.
2. You can write a separate class with a main method that has all your tests ("yays"). Remember to add JavaDocs to this OR remove this class before submission.
3. You're welcome to write JUnit tests if you know how to do that. Please don't submit these.

Warning: For this project you absolutely should test after every method you write and every change you make. You should even be thinking "How will I test this?" before writing a method, and you should be choosing the order of writing methods to help with testing.

We also noticed that a lot of students seem to be tracing code with print statements. That's a good starting place, but there are very nice tools called "debuggers" that you probably want to be aware of. One of the UTAs in CS310 a few years back made a video tutorial for VS Code, which you might find helpful:

- Running and compiling a code in VS Code for CS 310. - YouTube

And in a previous semester, another UTA found a bunch of tutorials and articles for using debuggers in other IDEs:

- Videos:
  - Debugging in IntelliJ - YouTube
  - Java Programming - How To Use The Eclipse Debugger Tools - YouTube
  - Debugging in jGrasp - YouTube
  - How to Debug your Java Program in Visual Studio Code (2020) - YouTube
- Articles by the IDE creators:
  - Debugging the Eclipse IDE for Java Developers | The Eclipse Foundation

- Tutorial: Debug your first Java application | IntelliJ IDEA (jetbrains.com)
- 06_Debugger.pdf (jgrasp.org)

Part 1: Squares

Squares are represented by the (x,y) position of the upper left corner, and their size. Squares also have a randomly generated color. This code is completely done for you except the following:

1. You need to be able to determine if a given (x,y) position is within the square's boundaries. This will, eventually, allow you to handle "clicks" on a given square.
2. You need to draw the squares.

Drawing: GUIs are actually very easy in Java, as there are multiple libraries that can help you. In this project we're going to use the Graphics class from java.awt (https://docs.oracle.com/javase/9/docs/api/java/awt/Graphics.html). Here are the methods you'll need to draw squares:

- setColor() -- https://docs.oracle.com/javase/9/docs/api/java/awt/Graphics.html#setColor-java.awt.Color-
- fillRect() -- https://docs.oracle.com/javase/9/docs/api/java/awt/Graphics.html#fillRect-int-int-int-int-
- drawRect() -- https://docs.oracle.com/javase/9/docs/api/java/awt/Graphics.html#drawRect-int-int-int-int-

Testing Suggestions: Definitely check ALL the methods in this class before continuing. The drawing you may want to test after Part 4 (when the GUI starts to work).

Part 2: Lists of Squares

Each window is going to need a list of squares. An outline of this class has been provided for you (along with some big-O requirements), and you are going to complete this provided code using linked lists!

You have two options for your linked lists in this project:

1. "Bare nodes" using the **`Node`** class we provided. This option means you'll be working directly with nodes throughout the project, rather than a linked list class.
2. A linked list class of ***your own*** design (no copying from anywhere!). If you decide to go this route, here are the rules you must follow:
    a. Your linked list class should be implemented as **`ThreeTenLinkedList`** in the provided file **`ThreeTenLinkedList.java`**. Note: there are also required static methods in this class which you will do in Part 5. Do not delete these!
    b. Your linked list must be made out of the nodes we defined in the provided **`Node`** class.
    c. Your implementation should not use "dummy nodes". All nodes in the linked list should contain "real" values from the list. Note: Java doesn't generally use "dummy nodes" like C/C++. C/C++ often uses dummy nodes to avoid passing around "double pointers", but Java has no such thing to avoid and we'd like you to try "the Java way".
    d. You will have learned and read about linked lists in the lecture and textbook, which means you will have seen code for linked lists before now, but as it says above, this should be ***your own*** design based on what you've learned. You still may not ***copy*** code from anywhere; that means no copy-and-paste but also no "manual copying" either.

It is highly recommended that you read the entire project description before you decide which of the above options you'll choose. If you choose the linked list class, you'll want to read over the project again to make sure you know which (of the many possible) linked list class methods you'll need.

There is no "right answer" to this. You can choose to implement this project either way as long as **`SquareList`** (and later **`WindowStack`**) work as expected. See **`SquareList.java`** and **`WindowStack.java`** for expected behavior.

Testing Suggestions: Make some squares, add them to the list, "pretend" someone has clicked on some (x,y) with **`handleClick()`**, print out the results using **`getHead()`** and a loop. Make different tests to simulate different scenarios,

such as squares being removed at the start of the list, squares being removed at the end of the list, squares being removed from the middle of the list, no squares being removed, all squares being removed.

Part 3: Windows

Windows are the coolest part of the project, so your professors wrote most of that for you. However, you do need to complete the following things:

1. You need to be able to determine if a given (x,y) position is within the window's boundaries. This will, eventually, allow you to handle "clicks" on a given window.
2. You need to track whether or not a window is the currently selected window. This is just a field that controls how windows are drawn (with or without a thick border). You'll be using this in Part 4.

Testing Suggestions: Tests similar to **`Square`**, but also you should use **`handleClick()`** to add to the list of squares and confirm that these show up in the list of squares (there is a **`getSquares()`** method for windows).

Part 4: Window Stacks

The window stack is actually even cooler than the windows themselves, so we left that to you. An outline of this class has been provided for you (along with some big-O requirements), and you are going to complete this provided code using linked lists. You can use your **`ThreeTenLinkedList`** or **`Node`** directly just like with **`SquareList`**.

Testing Suggestions: After this stage you should be able to see windows in the GUI, be able to draw squares, etc. The menus won't work until after Part 5, but you'll probably discover some bugs you need to fix. So do a lot of main method (and GUI) testing before you continue!

Note: If your GUI is "flickering", don't be worried; that just happens. The provided code redraws whenever you do something and, on some systems, the redraw might be slow enough that you see it happen. The code is not optimized for rendering images in any way, just drawing windows and squares on top of each other, which is not ideal. [Additionally, the AWT graphics library is very old, so it might not happen if we used a newer one like Swing of JavaFX.]

Part 5: Sorting Windows and Squares

While you were working on Parts 2-4, you probably came across bits of code related to sorting. You may also have wondered how the menu buttons were going to work. That's this part of the project. Sorting code is spread throughout the following:

- **`ThreeTenLinkedList`** – There are two static methods in this class, **`isSorted()`** and **`sort()`**. These are generic static methods that accept a **`NodePair`** (a head/tail combo variable) and a comparator. Here are some things that might be helpful when writing these:
  - Understanding Generic Methods: Generic Methods (The Java Tutorials > Bonus > Generics)
  - Comparable vs. Comparators: Object Ordering (The Java Tutorials > Collections > Interfaces)
  - Nested Classes: Nested Classes (The Java Tutorials > Learning the Java Language > Classes and Objects)

- **`WindowStack`** – The window stack needs to sort windows! Sorting by size is 99% done for you as a demonstration of how to write a comparator and call your **`sort()`** method from **`ThreeTenLinkedList`**. Sorting by location is on you; you'll have to do the same thing, but write a different comparator.

- **`SquareList`** – The square list will do the sorting of squares. Sorting by creation time (using the square's IDs) and sorting by the square's location in the window will be similar to how you did the sorting in **`WindowStack`**, but now for a list of squares.

There is *a lot* more information on sorting, suggested ways to sort linked lists, and requirements for the sorts in the **`ThreeTenLinkedList`** class.

Testing Suggestions: You can continue testing with the GUI (using the menus), but it will be easier to test some things with code. Examples: ensuring the sort is stable, ensuring the sort works for any linked list and comparator (e.g. one for integers), etc.

Requirements Summary

As always, you are required to:

- **Implement the classes as outlined** - You need to write the required classes and fill the provided template files.
- **JavaDocs** - You are required to write JavaDoc comments for all the required classes and methods.
- **Big-O** - Template files provided to you contains instructions on the REQUIRED Big-O runtime for many methods. Your implementation of those methods should NOT have a higher Big-O.
- **Style** - You are required to write using a consistent and readable coding style.

Additional requirements can be found in the grading rubric.

How To Handle a Multi-Week Project

While this project is given to you to work on over about two weeks, you are unlikely to be able to complete this is one weekend. We recommend the following schedule:

- Step 1 (Prepare): First Day (Friday 10/02)
  - Go over Project 2 description carefully.
  - Read about Linked Lists (Ch17 of textbook).
  - Read about Comparators (Ch4.8 of textbook)
  - Try out a debugger!
- Step 2 (`Square` + `SquareList`): First Weekend (Saturday 10/03-Sunday 10/04)
  - Implement and test methods of `Square`.
  - Implement and test methods of `SquareList` (except for Part 5)
- Step 3 (Help + `Window` + `WindowStack`): "School Work Week" (Monday 10/05-Thursday 10/08)
  - Get help with your `SquareList`.
  - Get help understanding Comparators.
  - Implement and test methods of `Window`.
  - Start implementing `WindowStack` (except for Part 5)
- Step 3 (Finish `WindowStack` + Part 5): Second Weekend (Friday 10/09-Sunday 10/11)
  - Finish implementing and testing methods of `WindowStack`.
  - Implement Part 5.
  - Test everything, a lot.
- Step 4 (`Wrapping-up`): Last Days (Monday 10/12-Tuesday 10/13)
  - Additional testing, debugging, get additional help.
  - ☺ Also, submit early for extra credit! See the rubric for additional information.

<table>
<tr><th>Monday</th><th>Tuesday</th><th>Wednesday</th><th>Thursday</th><th>Friday</th><th>Saturday</th><th>Sunday</th></tr>
<tr><td colspan="4"></td><td>First Day
(Prepare)</td><td colspan="2">First Weekend
(<code>Square</code> + <code>SquareList</code>)</td></tr>
<tr><td colspan="4">School Work Week
(Help + <code>Window</code> + <code>WindowStack</code>)</td><td colspan="3">Second Weekend
(Finish <code>WindowStack</code> + Part 5)</td></tr>
<tr><td colspan="2">Last Days
(<code>Wrapping-up</code>)</td><td>Late Day 1
(-10pts)</td><td>Late Day 2
(-20pts)</td><td colspan="3"></td></tr>
</table>